AGENT EVALUATION SCIENCE

Beyond Pass@k: Auditing Sampling Units and Acceptance Events in Coding-Agent Evaluation

Jiajun Jiang[1] Sharon Zheng[1] Natan Vidra[1] Spurthi Setty[1,*]

[1] Anote AI

[*] Corresponding author.

Abstract

The pass@k estimator of Chen et al. (2021) is unbiased only when its n inputs are independent full attempts and its success event is the one the evaluator cares about. We audit both choices for coding agents. First, we inspect the pass@k code of eight public harnesses and find that all of them count independent samples or attempts; the test-count misapplication we originally reported came from our own earlier harness, so we present it as an audit risk rather than common practice. Against the closed-form ground truth of a synthetic benchmark, that misapplication overstates reliability@5 by $+0.76$ to $+0.92$, while the rollout estimator stays within 0.012 of the truth. Second, we show that the acceptance event can decide a leaderboard. Rescoring 240 live generations from three model configurations, a binding zero-findings security predicate moves GPT-4o from first (1.00) to last (0.70) on reliability@5, and to 0.40 on security-sensitive tasks. Excluding rollouts that call an LLM API instead of implementing the function puts it back in first place, because 135 of 160 Claude-backed rollouts delegated in this way and 70 of those passed the tests. We give a six-field reporting checklist for acceptance events and sampling units. On 90 Claude Code rollouts over 30 SWE-bench Verified tasks, single-attempt resolved reliability is 0.689 but all-of-3 consistency is 0.533, and a regression-aware failure profile shows that most failures miss the required change without breaking existing tests. All evidence is small-scale; intervals are wide and we report them.

Keywords: Agent evaluation; pass@k; Reliability; Code security; Benchmark auditing

1 Introduction

Coding-agent leaderboards report pass@k, a resolve rate, or both. Behind each number sit two choices that are rarely written down: what counts as one *trial* and what counts as a *success*. The combinatorial estimator of Chen et al. (2021) is unbiased for the probability that at least one of k trials succeeds only if its n inputs are independent, identically distributed full attempts, and it answers only for the success event it was given.

An earlier version of this work (Jiang et al., 2026) argued that agentic harnesses misapply the estimator by passing unit-test counts as n. Reviewers made three objections. The claim named no harness that does this. The corrected metric was described as single-attempt reliability, which it is not. The security adjustment changed no score. This paper replaces the earlier argument with an audit and corrects each point:

- **A harness audit** (§2). All eight public harnesses we inspected count independent samples or attempts. The test-count error lived in our own CodeBench v1 code, so we treat it as a risk to check for, not a field-wide practice. The audit also surfaces quieter differences in aggregation and estimators.

- **Correct estimands and a common-estimand comparison** (§3). We separate single-attempt reliability p, any-of-k reliability $1-(1-p)^k$, and all-of-k consistency p^k, and score both estimators against the same closed-form ground truth.
- **The acceptance event decides the leaderboard** (§4). Rescoring stored live rollouts under four predefined acceptance events reorders three configurations twice, exposes a bug in our earlier per-pair security check, and reveals LLM-API delegation that tests alone accepted.
- **Reliability and a regression-aware failure profile on SWE-bench Verified** (§5): rel@1, rel@3 and pass^3 with intervals for Claude Code on 30 tasks with 3 attempts each, under two acceptance events.

2 What a Reliability Number Must Specify

We treat a reported reliability score as an answer to a question defined by six fields: (i) the *sampling unit*: what one of the n trials is; (ii) the *acceptance event*: the predicate a trial must satisfy; (iii) the *observed trial count* n per task; (iv) the *operational retry count* k, which should match deployment (one attempt, or k attempts with a selector that can recognise success); (v) the *aggregation*: macro over tasks, and which tasks are dropped; and (vi) the *uncertainty* reported. A score whose six fields are not stated cannot be compared with another score.

Audit. We read the pass@k code of eight public harnesses at the commits listed in Table 1 and recorded fields (i), (ii) and (v). Every harness uses an independent sample or attempt as its unit and a whole-attempt success event. None substitutes test counts. The test-count substitution appears only in our own CodeBench v1, whose function `_estimate_pass_at_k` called the estimator with $n =$ tests in one submission and $c =$ tests passed. We report this as a negative result for our earlier premise. The audit still finds differences that change what a number means. Terminal-Bench drops tasks with fewer than k attempts from the average. Its USACO adapter averages non-overlapping blocks of k attempts rather than using the unbiased estimator. τ-bench reports pass^k, the probability that *all* k trials succeed (Yao et al., 2025). SWE-bench reports one attempt per instance and counts a task as resolved only when all fail-to-pass and pass-to-pass tests pass (Jimenez et al., 2024).

3 Estimands, and a Common-Estimand Comparison

Let p be the probability that one independent full rollout of an agent on a task satisfies the acceptance event. Three quantities are often conflated:

$$\text{rel@}1 = p, \qquad \text{rel@}k = 1-(1-p)^k, \qquad \text{pass}^k = p^k. \tag{1}$$

With c of n rollouts accepted, the unbiased estimators are $\hat{p} = c/n$, $1-\binom{n-c}{k}/\binom{n}{k}$ (Chen et al., 2021), and $\binom{c}{k}/\binom{n}{k}$ (Yao et al., 2025). Our earlier version described rel@k as "the probability that a single deliberate attempt succeeds"; that is rel@1. For an agent run once in deployment, rel@1 and pass^k answer the operational question. rel@k fits settings where k attempts are made and success can be recognised. We report rel@1 alongside rel@5 throughout.

Why test counts are not trials. A single submission that passes 4 of 10 tests and another that passes 2 of 5 have the same pass rate, 0.40. Read as (n, c), they give pass@5 scores of 0.976 and 1.000, so the score moves with suite size alone. This shows only that the substitution is not invariant to suite size. It does not show that the two agents have equal reliability.

Common-estimand comparison. The earlier version compared a test-level quantity with a rollout-level one, so its "inflation" mixed two estimands. Here both estimators target the

Harness (commit)	Unit n	Acceptance event; aggregation note
HumanEval (Chen et al., 2021) (b9453e8)	completions	all tests pass; k reported only if every task has $n \geq k$
EvalPlus (Liu et al., 2023) (26d6d00)	samples	base (and plus) tests pass; same $n \geq k$ guard
BigCodeBench (Zhuo et al., 2025) (09dd993)	samples	status `PASS`
LiveCodeBench (Jain et al., 2024) (28fef95)	generations	all test cases pass
bigcode-eval-harness (BigCode Project, 2022) (8fc5bae)	completions	all tests pass
Terminal-Bench (The Terminal-Bench Team, 2025) (d28711d)	attempts	task resolved; tasks with $n < k$ dropped; USACO adapter uses block averaging
τ-bench (Yao et al., 2025) (59a200c)	trials	reward success; reports pass^k (all-of-k)
SWE-bench (Jimenez et al., 2024) (02e7a74)	1 attempt	all F2P and P2P tests pass; resolve rate
CodeBench v1 (ours)	**unit tests**	**test passes** (misapplied)

Table 1 | Sampling unit and acceptance event in eight public harnesses and in our earlier code. Only our own v1 harness used tests as the sampling unit.

same quantity. In the CodeBench simulator, each (task, configuration) pair has a latent skill $s \sim U(0.30, 1.0)$ and spread σ, a rollout's pass rate is a clamped Gaussian around s, and a rollout is accepted when its pass rate is at least 0.95. True single-rollout success is therefore $q = 1 - \Phi((0.95 - s)/\sigma)$, and true rel@$5 = 1 - (1 - q)^5$ in closed form. We replay the simulator's random stream to recover (s, σ) for every pair. Table 2 scores both estimators against this truth for the original design (10 tasks, 8 rollouts) and a larger one (200 tasks, 20 rollouts). The test-count estimator reports about 0.96–0.98 whatever the truth. The rollout estimator tracks the truth and recovers its configuration ranking in both designs.

	10 tasks × 8 rollouts			200 tasks × 20 rollouts		
Profile	True	Test-count	Rollout	True	Test-count	Rollout
P1	0.068	0.975	0.125	0.248	0.983	0.240
P2	0.033	0.965	0.000	0.191	0.977	0.173
P3	0.029	0.959	0.000	0.222	0.979	0.213
Mean bias vs. truth		$+0.922$	-0.002		$+0.760$	-0.012
Mean abs. error		0.922	0.049		0.760	0.047

Table 2 | reliability@5 against closed-form ground truth in the CodeBench simulator (seed 42). Bias and error are averaged over (task, profile) pairs. P1–P3 are synthetic profiles, not real systems.

Single-rollout proxy, revisited. The earlier H3 result ($\rho = 0.417$ over 30 pairs) correlated a proxy with an estimated target that was zero for 28 of 30 pairs. It also took the proxy's tool-use term from a different rollout than its pass rate. With both fixed and the proxy scored against the closed-form truth, $\rho = 0.19$ (95% bootstrap CI $[-0.20, 0.49]$) on the 30-pair design. On the 600-pair design it is $\rho = 0.49$ $[0.43, 0.55]$. In the larger design, 50% of pairs still have true rel@$5 < 0.01$, so the target remains heavily concentrated near zero. We read this as weak evidence that this particular proxy is an inadequate surrogate in this simulator. It is not evidence about single-rollout proxies in general.

4 The Acceptance Event Decides the Ranking

Setup. The earlier version generated 240 rollouts through live APIs: 3 configurations $\times$ 10 tasks $\times$ 8 independent rollouts. Five tasks are algorithmic (merge sort, LRU cache, Dijkstra, knapsack, streaming median). Five are security-sensitive and invite unsafe implementations (expression evaluation, snippet execution, shell commands, YAML loading, dynamic import). Each task has 5 unit tests. The configurations are not products. S-Anote and S-CC are both `claude-sonnet-4-6`, with system prompts introducing it as "Anote Code" and "Claude Code" respectively. GPT-4o is `gpt-4o` with a generic assistant prompt. We stored every rollout's code, so all results below are rescorings with no new generations.

Two problems in the earlier analysis. First, the earlier security check looked up one submission per (task, configuration) pair, the last rollout's, and applied it to all 8 rollouts. Second, its threshold of a score ≥ 0.80, with each of 7 lexical patterns costing $1/7$, accepted a rollout with one high-severity finding. Together these explain why the adjustment changed no score even though GPT-4o had 24 functionally passing rollouts with findings. Here every rollout is checked on its own code.

Acceptance events. We fix four events before looking at rankings: A0, all tests pass; A1, A0 and zero scanner findings (the predicate the earlier abstract described); A2, A0 and the code does not import an Anthropic or OpenAI client; A3, A1 and A2. We added A2 after inspecting rollouts. 135 of the 160 Sonnet-backed rollouts (S-Anote 55/80, S-CC 80/80) answer the task by calling an LLM API from inside the requested function rather than implementing it. The test runner executed generated code with network access and API credentials available, so 70 of these delegating rollouts passed (S-Anote 34, S-CC 36). No GPT-4o rollout delegated.

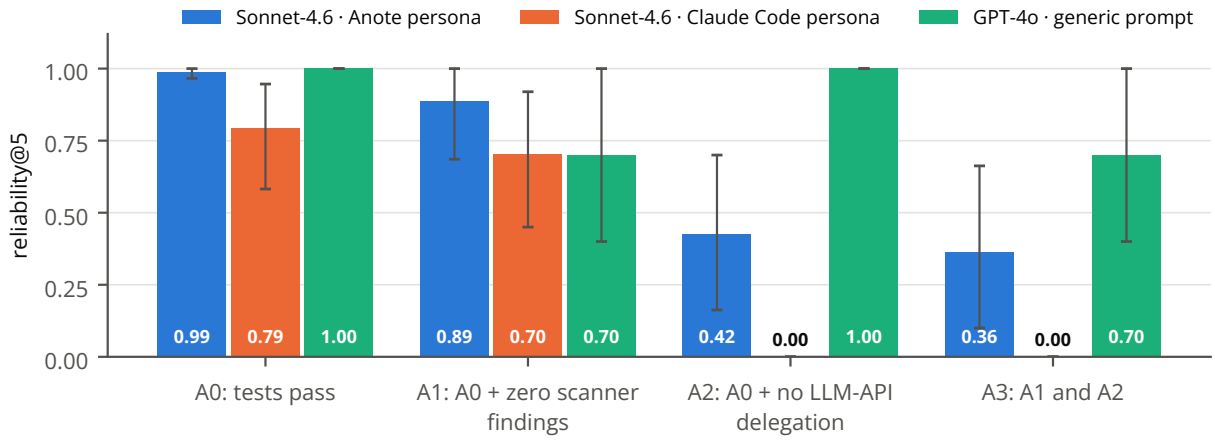


Figure 1 | reliability@5 for the same 240 stored rollouts under four acceptance events. Error bars are 95% bootstrap intervals over the 10 tasks. Rankings change between A0 and A1 and again between A1 and A2.

Results. Under A0, GPT-4o leads (1.000). Under A1 it falls to last (0.700, against 0.887 for S-Anote and 0.704 for S-CC). Every one of its rollouts on the expression, snippet and shell tasks passes the tests through bare `eval`, `exec` or `shell=True`. On the five security-sensitive tasks alone, its rel@5 drops from 1.000 to 0.400, while the Sonnet configurations drop from 0.975 to 0.775 and from 0.957 to 0.779. The A1 reordering is not statistically secure with 10 tasks: in a paired bootstrap over tasks, GPT-4o trails S-Anote in 89.5% of resamples. Under A2 and A3, GPT-4o leads again and S-CC scores zero, because every one of its passing rollouts delegated. The 0.195 gap between the two Sonnet personas under A0, which the earlier version

	S-Anote		S-CC		GPT-4o	
Acceptance event	rel@1	rel@5	rel@1	rel@5	rel@1	rel@5
A0 tests pass	0.738	0.988	0.450	0.793	1.000	1.000
A1 + zero findings	0.675	0.887	0.425	0.704	0.700	0.700
A2 + no delegation	0.312	0.425	0.000	0.000	1.000	1.000
A3 A1 and A2	0.300	0.362	0.000	0.000	0.700	0.700

Table 3 | Single-attempt and any-of-5 reliability over all 10 tasks. Intervals for rel@5 are in Figure 1.

attributed to persona framing in general, is largely this delegation behaviour. The answer to "which configuration is most reliable?" therefore depends on an acceptance event that leaderboards seldom state. Security screening and delegation screening are two predicates within that one event, not separate metrics.

Relation to secure-pass@k. DeepGuard's secure-pass@k already applies the Chen estimator to a joint functional-and-security success event (Huang et al., 2026). A1 is the same construction, and we do not claim it as a new metric. Our point concerns specification. Whether the security predicate is checked per rollout, whether it binds at one finding, and what other conditions sit in the acceptance event each changed the ranking here. Industry reports of vulnerable AI-generated code that still passes functional tests (Veracode, 2025) motivate checking these choices.

5 Case Study: Reliability and Failure Profile on SWE-bench

Setup. We ran Claude Code CLI (v2.1.305, model `claude-sonnet-5-5`) headless in `acceptEdits` mode, so the agent can read and edit files but has no shell and cannot run tests. The sample is 30 SWE-bench Verified tasks drawn with one seed, stratified across 12 repositories, and excluding the five tasks of our earlier pilot (Appendix Table B2). Each task received 3 independent attempts in fresh checkouts, 90 rollouts in all, scored with the official harness. The agent saw only the issue text, repository and base commit. Every rollout is counted, and 0 harness errors remained. We had planned a second model and five attempts per task, but cut both to fit our compute budget. This section therefore compares acceptance events for one configuration and cannot test whether they reorder configurations. With 3 attempts we report rel@3 in place of rel@5.

Config	Acceptance event	rel@1	rel@3	pass^3
sonnet	F2P pass	0.733 [0.59, 0.87]	0.833 [0.70, 0.97]	0.600 [0.43, 0.77]
sonnet	Resolved	0.689 [0.54, 0.82]	0.800 [0.67, 0.93]	0.533 [0.37, 0.70]

Config	Resolved	Fixed, regressed	Missed, clean	Missed, regressed	No usable patch	Harness error
sonnet	62	4	16	4	4	0

Table 4 | Claude Code on 30 SWE-bench Verified tasks, 3 attempts each. Top: reliability under fail-to-pass (F2P) and resolved acceptance, with 95% task-bootstrap intervals. Bottom: outcome of each of the 90 rollouts.

Results. Under F2P acceptance $\widehat{\text{rel@1}} = 0.733$ $[0.59, 0.87]$, and under the official resolved event it is 0.689 $[0.54, 0.82]$ (Table 4). The gap is the 4 rollouts that made the failing tests pass but broke a previously passing test. The intervals overlap almost entirely, so we do not claim that the two events differ for this configuration. The estimands differ more. At least one of

three attempts resolved 24 of the 30 tasks, giving rel@$3 = 0.800$ $[0.67, 0.93]$, but the share of tasks on which all three attempts resolved is only pass$^3 = 0.533$ $[0.37, 0.70]$. For an agent run once without a selector, rel@1 and pass^k answer the operational question, and rel@k alone would overstate it.

Failure profile. Of the 90 rollouts, 16 missed the required change while keeping every existing test passing, and 4 missed it and broke existing tests. The remaining 4 patches applied but crashed the test run, either by omitting a function the hidden tests import or by raising an error that stopped the test runner. The harness could not grade them, and we count them as failures. In this sample, an agent that cannot run tests fails mostly by under-fixing rather than by breaking behaviour. A resolve rate merges these failure types, which matter differently in deployment, so we suggest reporting the counts beside rel@1.

6 Limitations

The audit covers eight harnesses at single commits and records how each computes its metric, not how leaderboards built on them are reported. The simulator results show how the estimators behave on a known data-generating process; they say nothing about production agents. The live-rollout study has 3 configurations and 10 tasks, so its intervals are wide, and two of the configurations share a model. Its security scanner matches 7 lexical patterns. It cannot see most vulnerability classes, and its regex for `eval` would also match the safe `literal_eval` (we confirmed no such false positive in these data). Some security tasks invite flagged primitives in their specifications. The delegation check is also lexical, and A2 was defined after inspecting rollouts. The SWE-bench study uses one agent harness and one model, 30 tasks drawn with one sampling seed, 3 attempts per task, and `acceptEdits` mode with no shell. Its intervals are wide, and with one configuration it cannot show whether the acceptance event reorders configurations on SWE-bench. Its harness ran under x86_64 emulation on Apple silicon.

7 Conclusion

The estimator of Chen et al. (2021) works when its inputs are independent attempts. The public harnesses we audited supply that, and our own earlier harness did not. The larger risk we found is quieter: the acceptance event. With no new generations, changing only that predicate reordered three configurations twice, exposed a per-pair bug in our own security adjustment, and revealed rollouts that passed tests by calling an LLM. On SWE-bench Verified, one Claude Code configuration has single-attempt resolved reliability 0.689 but all-of-3 consistency 0.533, and most of its failures left existing tests intact. We recommend that any reported reliability number state the six fields of §2, give rel@1 beside any rel@k, and attach uncertainty. Code, stored rollouts and rescoring scripts are at https://github.com/anote-ai/research-codebench.

Prior Version

An earlier version of this work is available as Jiang et al. (2026). This version replaces its central empirical claims as described in §1.

Ethics Statement

Scores here describe specific model configurations on small task sets and should not be read as rankings of commercial products; we renamed configurations accordingly. A lexical scanner is not a security audit. Generated code was executed with network access and API credentials, which let delegating code run. Harnesses should sandbox execution and withhold credentials.

Generative AI Use

LLM coding agents are the subjects of the evaluation. Generative AI tools also assisted with drafting text and analysis code. The authors reviewed and verified every result and take full responsibility for the content.

References

BigCode Project. A framework for the evaluation of code generation models. https://github.com/bigcode-project/bigcode-evaluation-harness, 2022. Commit 8fc5bae, accessed October 2026. Cited at: 3.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*. Cited at: 1, 2, 6, 5, 7.

Li Huang, Zhongxin Liu, Yifan Wu, Tao Yin, Dong Li, Jichao Bi, et al. DeepGuard: Secure code generation via multi-layer semantic aggregation. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 19793–19813, 2026. Cited at: 1.

Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, et al. 2024. LiveCodeBench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*. Cited at: 3.

Jiajun Jiang, Sharon Zheng, Natan Vidra, and Spurthi Setty. 2026. Beyond pass@k: Measuring reliability and security of agentic code generation. *arXiv preprint arXiv:2608.14711*. Cited at: 1, 2.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, et al. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. Cited at: 1, 4.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems*, volume 36, 2023. Cited at: 3.

The Terminal-Bench Team. Terminal-bench: A benchmark for AI agents in terminal environments. https://github.com/laude-institute/terminal-bench, 2025. Commit d28711d, accessed October 2026. Cited at: 3.

Veracode. 2025 GenAI code security report. https://www.veracode.com/resources/analyst-reports/2025-genai-code-security-report/, 2025. Cited at: 1.

Shunyu Yao, Noah Shinn, Pedram Razavi, and Karthik Narasimhan. τ-bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations*, 2025. Cited at: 1, 5, 4.

Terry Yue Zhuo et al. BigCodeBench: Benchmarking code generation with diverse function calls and complex instructions. In *International Conference on Learning Representations*, 2025. Cited at: 3.

A Evaluation Protocol

Field	§3 simulator	§4 live rollouts	§5 SWE-bench
Sampling unit	simulated rollout	one API generation	one Claude Code run
Acceptance event	pass rate ≥ 0.95	A0–A3 (§4)	F2P pass; resolved (all F2P and P2P pass)
Trials per task n	8 or 20	8	3
Retry count k	5 (and 1)	5 and 1	1 and 3 (pass^3)
Aggregation	macro over pairs	macro over 10 tasks	macro over 30 tasks (12 repos)
Uncertainty	bias, MAE; bootstrap CI for ρ	task bootstrap (10,000)	task bootstrap (10,000)

Table A1 | The six reporting fields of §2 for each study in this paper.

SWE-bench run details. Agent: Claude Code CLI v2.1.305 (`claude -p`, `--permission-mode acceptEdits`, `--model sonnet`), resolved to model id `claude-sonnet-5-5` in every rollout. Each attempt had a 1800 s timeout and a $5 budget cap; no attempt reached either, and agent cost was $9.46 in total. Tasks were drawn by `scripts/prepare_swebench_sample.py` with seed 0, stratified by repository with at most four per repository, excluding the pilot tasks; the task ids are in data/swebench_aes30_ids.txt. Scoring used the official harness (`swebench` 5.0.2) with the published x86_64 evaluation images, run under emulation on an Apple-silicon Mac. Three attempts on one task failed within seconds on an account usage limit before the model did any work; we set those records aside and re-ran them, and the logs record this. No other attempt was re-run.

B Additional Results

	Passing rollouts (of 8)			Passing with a finding		
Task	S-Anote	S-CC	GPT-4o	S-Anote	S-CC	GPT-4o
expression evaluation	8	8	8	0	0	8 (`eval`)
snippet execution	3	2	8	3	2	8 (`exec`)
shell command	8	7	8	1	0	8 (`shell=True`)
YAML loading	2	2	8	0	0	0
dynamic import	4	6	8	1	0	0
5 algorithmic tasks	34	11	40	0	0	0

Table B1 | Per-task counts behind Table 3. Every finding on a passing rollout occurs on a security-sensitive task.

Reproduction. analysis/synthetic_common_estimand.py recomputes Table 2 and the proxy correlations. analysis/h4_rescore.py recomputes Tables 3 and B1 from the stored data/h4_results.json. analysis/plot_ladder.py draws Figure 1. The scripts use fixed seeds: 42 for the simulator, 0 and 1 for the bootstraps.

Instance	Difficulty	Resolved	F2P	P2P	Regressions
astropy__astropy-12907	medium	yes	2/2	13/13	0
astropy__astropy-13033	medium	no	0/1	20/20	0
astropy__astropy-13236	medium	no	0/2	644/644	0
astropy__astropy-13398	hard	no	0/4	63/68	5
astropy__astropy-13453	medium	no	0/1	2/9	7

Table B2 | Earlier SWE-bench Verified pilot, not part of the §5 sample: Claude Code v2.1.185 in `acceptEdits` mode, one attempt on each of the first five Verified instances. One was resolved ($\widehat{\text{rel@1}} = 0.20$, Wilson 95% interval $[0.04, 0.62]$). F2P and P2P give passed/total hidden fail-to-pass and pass-to-pass tests.